Conditional Text Generation

NTV 2026 / HC5b

Today's lecturer: Wilker Aziz w.aziz@uva.nl

Outline

This class is about **conditional text generation**. That is, problems where we have two pieces of text, one is given as input the other is expected as output, which are related by some linguistic relation.

For example, in translation, the input we condition on is text in a *source* language and the output we are expected to generate is text in a *target* language, the two texts are related for they convey the same message (they are 'semantically' or 'translationally' equivalent).

Suggested reading:

- Chapter 12 of textbook

Goals

ILOs. After this class the student can

- recognise sequence modelling problems as instances of conditional language modelling
- parameterise sequence-to-sequence models using techniques such as recurrent encoder-decoders
- optimise sequence-to-sequence models via maximum likelihood estimation
- predict an output sequence using a search algorithm

Table of contents

1. Sequence modelling
2. Parameterisation
3. Parameter estimation
4. Predictions
5. Design Choices
6. Limitations
7. Puzzle

Sequence modelling

Sequence prediction problems

Many NLP tasks involve conditioning on text and predicting sequences.

In some cases the input and the output sequences are "step-aligned":

- part-of-speech tagging [10]
- named-entity recognition [9]

But, in general, there may be no obvious alignment amongst positions of the input and the output sequences. The task-specific relationship that makes the two sequences ideally related to one another need not be expressible in terms of small units:

- machine translation [13]
- text summarisation [11]
- entity retrieval [1]
- information extraction [7]

Conditional language models

Deploying a system for any of these tasks (translation, summarisation, question answering, etc.) requires a lot of expert knowledge (about task, datasets, design decisions, etc.), but most solutions employ a similar *modelling* backbone: a conditional language model.

Modelling a *sequence-to-sequence* mapping

We are interested in modelling a specific relationship between pairs of sequences:

- an input sequence x from an input space $\mathcal{X}$
- an output sequence y from an output space $\mathcal{Y}$

We will assume this relationship can be modelled directionally ($x \rightarrow y$) in a non-deterministic way.[1]

[1] **Notation** capital letters for random variables (e.g., Y), lowercase letters for their assignments (e.g., y), calligraphic letters for sample spaces (e.g., $\mathcal{Y}$). We use Y_j to denote a step in a random sequence and $Y_{<j}$ to denote a prefix sequence (up until but not including the Y_j). P_Y is the distribution of Y, $P_{Y|X=x}$ is the distribution of Y given $X = x$. $P(Y = y|X = x)$ is the probability of observing $Y = y$ given $X = x$.

Probabilistic modelling

We will treat y as an observation for a random variable (rv) Y, which we draw conditionally given an observation x for the rv X.

The probability $P(Y = y|X = x)$ with which we observe $Y = y$ conditioned on $X = x$ is given by a parametric function with parameters θ:

$$P(Y = y|X = x) = f(y|x; \theta)$$

We do not prescribe a generative story for X itself (that is to say we do not model the joint distribution P_{XY}).

Probabilistic modelling

We will treat y as an observation for a random variable (rv) Y, which we draw conditionally given an observation x for the rv X.

The probability $P(Y = y|X = x)$ with which we observe $Y = y$ conditioned on $X = x$ is given by a parametric function with parameters θ:

$$P(Y = y|X = x) = f(y|x; \theta)$$

We do not prescribe a generative story for X itself (that is to say we do not model the joint distribution P_{XY}).

Our first job, as modellers, is to design this probability mass function (pmf). Once it is in place, we will discuss how to estimate parameters for it, and, finally, how to use it to make predictions.

Challenges

Designing a pmf involves

1. specifying the parametric family
2. picking a value for its parameters

Let's concentrate on (1), assuming that we will be employing a form of gradient-based optimisation for (2).

Parameterisation

Conditional probability distributions (cpds) for structures

Given any $x \in \mathcal{X}$, we want to be able to parameterise a distribution over outcomes of Y. There are 2 key challenges here:

- x is a complex data structure (a variable-length sequence of discrete symbols), from an unbounded set;
- any instance of y is also a complex data structure (a variable-length sequence of discrete symbols), also from an unbounded set.

To overcome these challenges we will employ

Conditional probability distributions (cpds) for structures

Given any $x \in \mathcal{X}$, we want to be able to parameterise a distribution over outcomes of Y. There are 2 key challenges here:

- x is a complex data structure (a variable-length sequence of discrete symbols), from an unbounded set;
- any instance of y is also a complex data structure (a variable-length sequence of discrete symbols), also from an unbounded set.

To overcome these challenges we will employ

- a neural *encoder* for representing input sequences
- a neural *decoder* for the (probabilistic) generation of output sequences in a autoregressively factorised manner

Autoregressive factorisation

We exploit a decomposition of output sequences into parts, where each part is drawn from a ‘small’ outcome space of known ‘target symbols’.

Autoregressive factorisation

We exploit a decomposition of output sequences into parts, where each part is drawn from a 'small' outcome space of known 'target symbols'.

For example, a POS tag sequence can be decomposed into a sequence of *word categories*:

$$x = \langle \mathrm{I}, \mathrm{am}, \mathrm{going}, \mathrm{home} \rangle$$
$$y = \langle \mathrm{PRP}, \mathrm{VBP}, \mathrm{VBG}, \mathrm{NN} \rangle$$

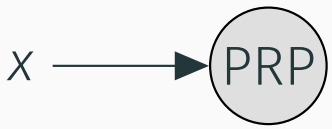
x

Autoregressive factorisation

We exploit a decomposition of output sequences into parts, where each part is drawn from a 'small' outcome space of known 'target symbols'.

For example, a POS tag sequence can be decomposed into a sequence of *word categories*:

$$x = \langle \text{I}, \text{am}, \text{going}, \text{home} \rangle$$
$$y = \langle \text{PRP}, \text{VBP}, \text{VBG}, \text{NN} \rangle$$

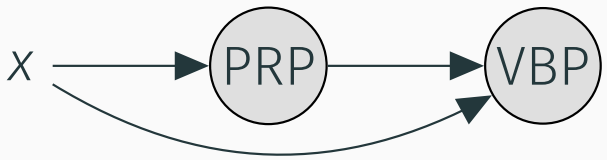


Autoregressive factorisation

We exploit a decomposition of output sequences into parts, where each part is drawn from a 'small' outcome space of known 'target symbols'.

For example, a POS tag sequence can be decomposed into a sequence of *word categories*:

$$x = \langle \text{I}, \text{am}, \text{going}, \text{home} \rangle$$
$$y = \langle \text{PRP}, \text{VBP}, \text{VBG}, \text{NN} \rangle$$

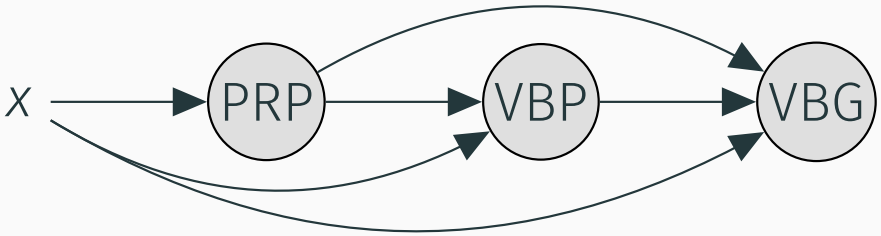


Autoregressive factorisation

We exploit a decomposition of output sequences into parts, where each part is drawn from a 'small' outcome space of known 'target symbols'.

For example, a POS tag sequence can be decomposed into a sequence of *word categories*:

$$x = \langle \text{I}, \text{am}, \text{going}, \text{home} \rangle$$
$$y = \langle \text{PRP}, \text{VBP}, \text{VBG}, \text{NN} \rangle$$

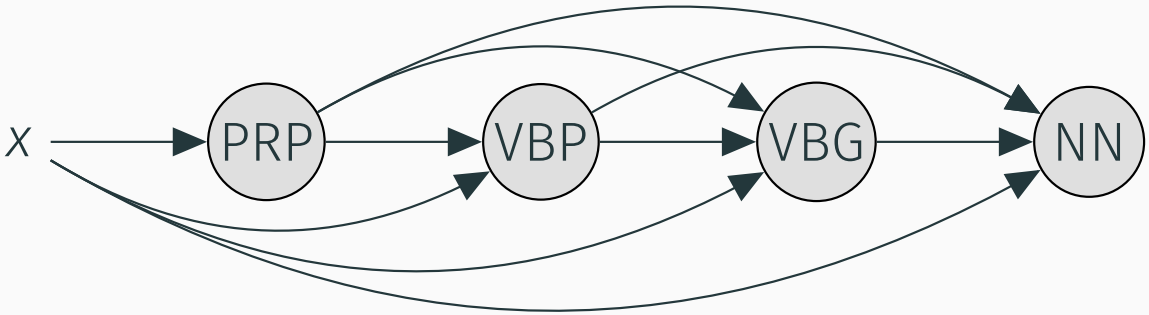


Autoregressive factorisation

We exploit a decomposition of output sequences into parts, where each part is drawn from a 'small' outcome space of known 'target symbols'.

For example, a POS tag sequence can be decomposed into a sequence of *word categories*:

$$x = \langle \text{I}, \text{am}, \text{going}, \text{home} \rangle$$
$$y = \langle \text{PRP}, \text{VBP}, \text{VBG}, \text{NN} \rangle$$



Another example

A translation can be decomposed into a sequence of *target words*:

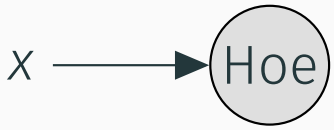
$x = \langle \text{How}, \text{are}, \text{you}, \text{doing}, ? \rangle$
$y = \langle \text{Hoe}, \text{gaat}, \text{het}, ? \rangle$

x

Another example

A translation can be decomposed into a sequence of *target words*:

$x = \langle \text{How}, \text{are}, \text{you}, \text{doing}, ? \rangle$
$y = \langle \text{Hoe}, \text{gaat}, \text{het}, ? \rangle$



Another example

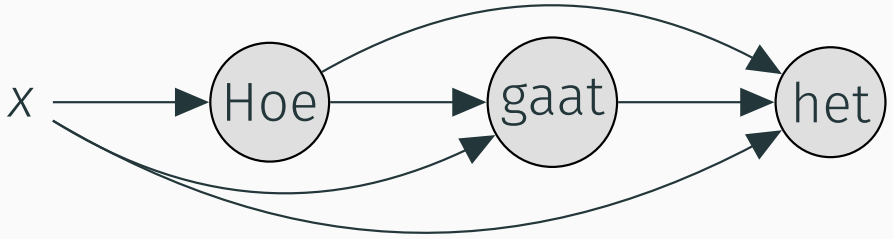
A translation can be decomposed into a sequence of *target words*:

$x = \langle \text{How}, \text{are}, \text{you}, \text{doing}, ? \rangle$
$y = \langle \text{Hoe}, \text{gaat}, \text{het}, ? \rangle$

Another example

A translation can be decomposed into a sequence of *target words*:

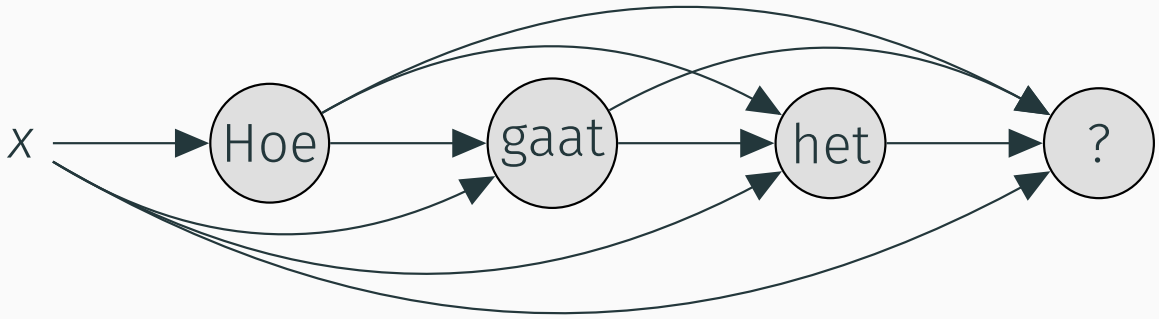
$x = \langle \text{How}, \text{are}, \text{you}, \text{doing}, ? \rangle$
$y = \langle \text{Hoe}, \text{gaat}, \text{het}, ? \rangle$



Another example

A translation can be decomposed into a sequence of *target words*:

$x = \langle \text{How}, \text{are}, \text{you}, \text{doing}, ? \rangle$
$y = \langle \text{Hoe}, \text{gaat}, \text{het}, ? \rangle$



Yet another example

A summary can be decomposed into a sequence of *words* (from the same vocabulary as those of the input):

$x = \langle$NTV, is, a, thorough, account, of, the, technical, developments, in, the, field, of, NLP, connecting, statistics, PGMs, ML, NNs, and, linguistics, to, tackle, real-world, AI applications, .$\rangle$

$y = \langle \text{NTV}, \text{rocks}, ! \rangle$

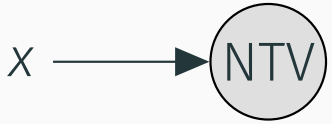
x

Yet another example

A summary can be decomposed into a sequence of *words* (from the same vocabulary as those of the input):

$x = \langle$NTV, is, a, thorough, account, of, the, technical, developments, in, the, field, of, NLP, connecting, statistics, PGMs, ML, NNs, and, linguistics, to, tackle, real-world, AI applications, .$\rangle$

$y = \langle \text{NTV}, \text{rocks}, ! \rangle$

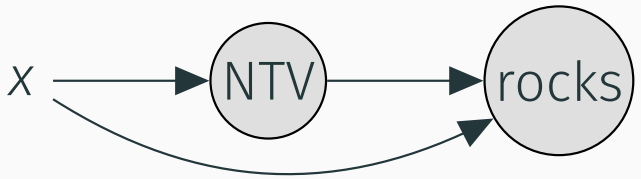


Yet another example

A summary can be decomposed into a sequence of *words* (from the same vocabulary as those of the input):

$x = \langle$NTV, is, a, thorough, account, of, the, technical, developments, in, the, field, of, NLP, connecting, statistics, PGMs, ML, NNs, and, linguistics, to, tackle, real-world, AI applications, .$\rangle$

$y = \langle \text{NTV}, \text{rocks}, ! \rangle$

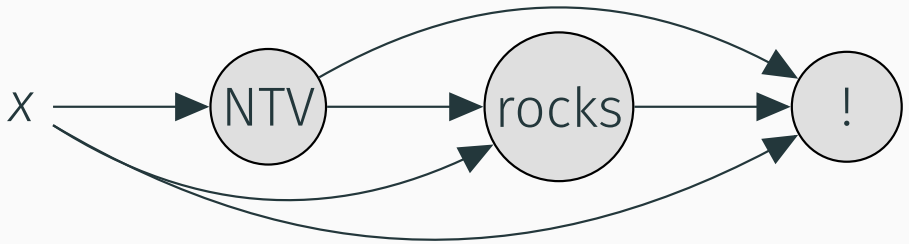


Yet another example

A summary can be decomposed into a sequence of *words* (from the same vocabulary as those of the input):

$x = \langle$NTV, is, a, thorough, account, of, the, technical, developments, in, the, field, of, NLP, connecting, statistics, PGMs, ML, NNs, and, linguistics, to, tackle, real-world, AI applications, .$\rangle$

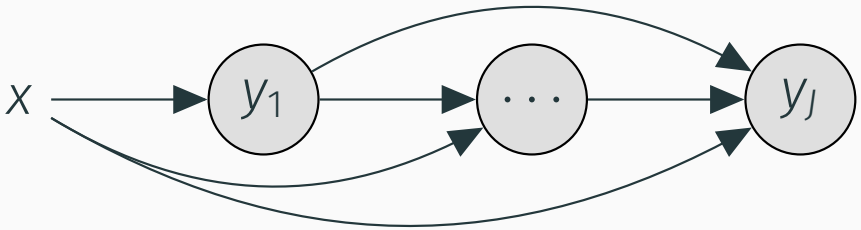
$y = \langle \text{NTV}, \text{rocks}, ! \rangle$



In general

For an input-output pair:

$$x = \langle x_1, \ldots, x_I \rangle$$
$$y = \langle y_1, \ldots, y_J \rangle$$



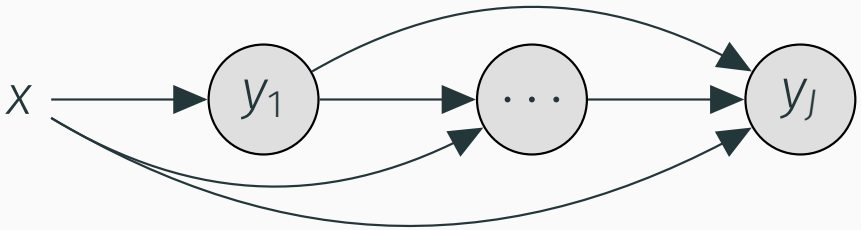
$$P(Y = y | X = x) = \prod_{j=1}^{J} P(Y_j = y_j | \underbrace{X = x, Y_{<j} = y_{<j}}_{\text{parents of } j\text{th rv}}) \tag{1}$$

In the decomposition, conditioned on increasingly complex context, each part is drawn from a small sample space. Models factorised this way are often referred to as *autoregressive*.

In general

For an input-output pair:

$$x = \langle x_1, \ldots, x_I \rangle$$
$$y = \langle y_1, \ldots, y_J \rangle$$



$$P(Y = y | X = x) = \prod_{j=1}^{J} P(Y_j = y_j | \underbrace{X = x, Y_{<j} = y_{<j}}_{\text{parents of } j\text{th rv}}) \tag{1}$$

In the decomposition, conditioned on increasingly complex context, each part is drawn from a small sample space. Models factorised this way are often referred to as *autoregressive*.

You can see that, if not for the presence of an additional input x, this is basically an autoregressive language model, and that's why we call it a **conditional language model**.

One C-way classifier, J steps

A general model of sequence prediction is in fact obtained by repeated application of something very similar to a shared text classifier:

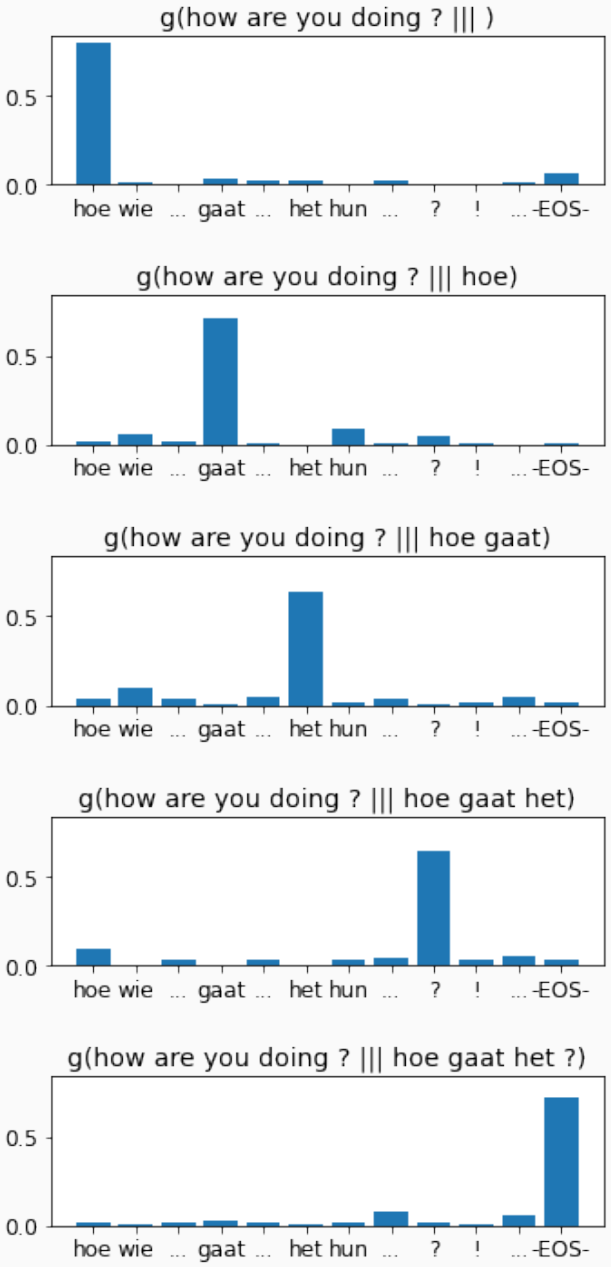
$$Y_j | \underbrace{X = x, Y_{<j} = y_{<j}}_{\text{conditioning context}} \sim \mathrm{Cat}(\underbrace{\mathbf{g}(x, y_{<j}; \theta)}_{\text{outcome probs}}) \tag{2}$$

Here, $\mathbf{g}$ maps from an input x and an partial output $y_{<j}$ to the probabilities of the possible outcomes for the jth step. The outcome space of each step is finite, relatively small, and fixed across steps (e.g., all tags, all words, all subword units, etc.).

Translation example

Same NN $\mathbf{g}(\cdot;\theta)$ reused over and over, each time mapping from a growing context (x and $y_{<j}$) to a probability distribution over the same categorical space (i.e., space of words).

In translation the output length is not determined by the length of the input, instead we repeat this process until a special terminating symbol is observed or generated.



A neural translation model

Statistical model let the function $\mathbf{g}$ map from an input x and prefix $y_{<j}$ to a distribution over V words:

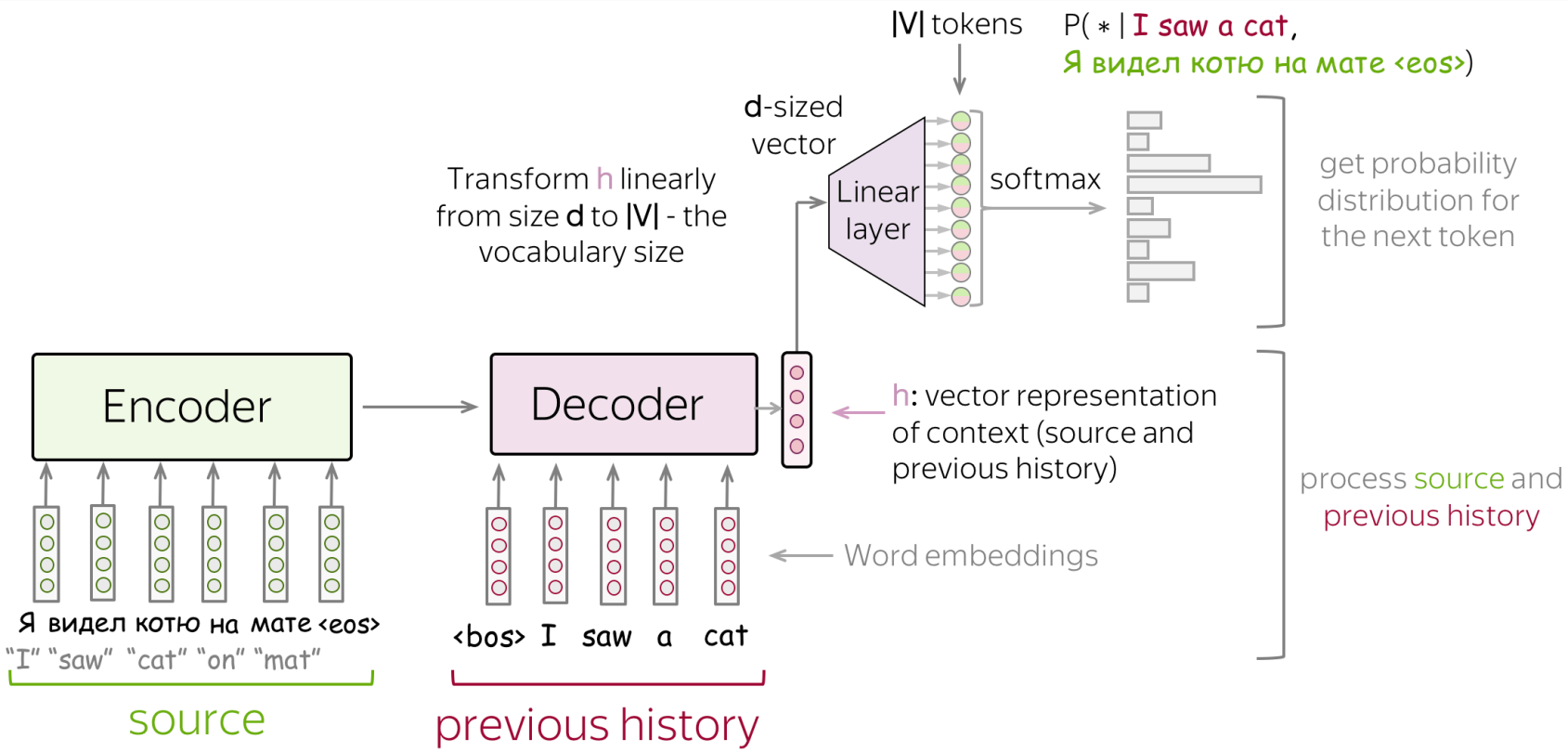
$$Y_j | X = x, Y_{<j} = y_{<j} \sim \mathrm{Cat}(\mathbf{g}(x, y_{<j}; \theta)) \tag{3}$$

[2] Figure from Lena Voita's NLP course for You. See animation.

A neural translation model

Statistical model let the function $\mathbf{g}$ map from an input x and prefix $y_{<j}$ to a distribution over V words:

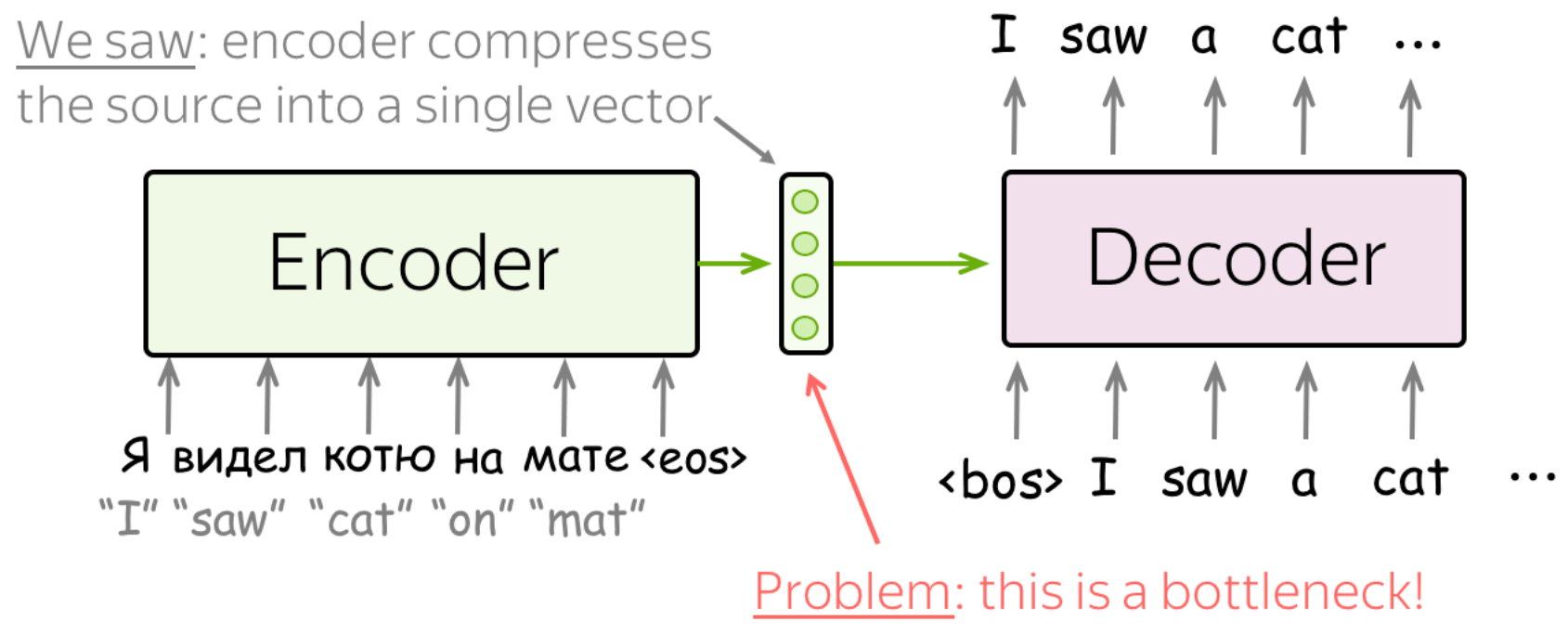
$$Y_j | X = x, Y_{<j} = y_{<j} \sim \mathrm{Cat}(\mathbf{g}(x, y_{<j}; \theta)) \tag{3}$$



[2] Figure from Lena Voita's NLP course for You. See animation.

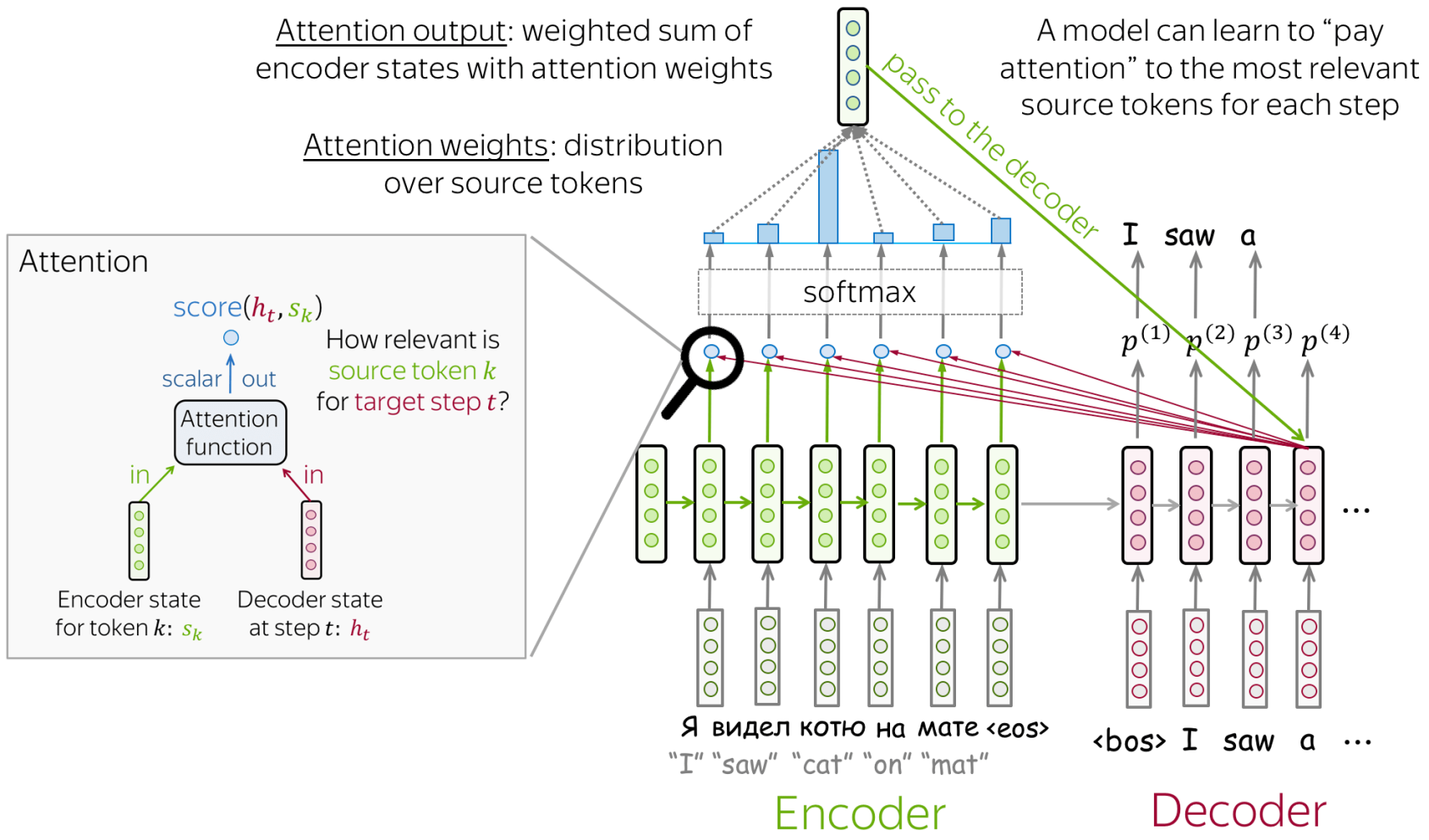
A better encoder-decoder model...

The basic encoder-decoder model represents the input sequence once and reuses that fixed representation for all decoding steps.



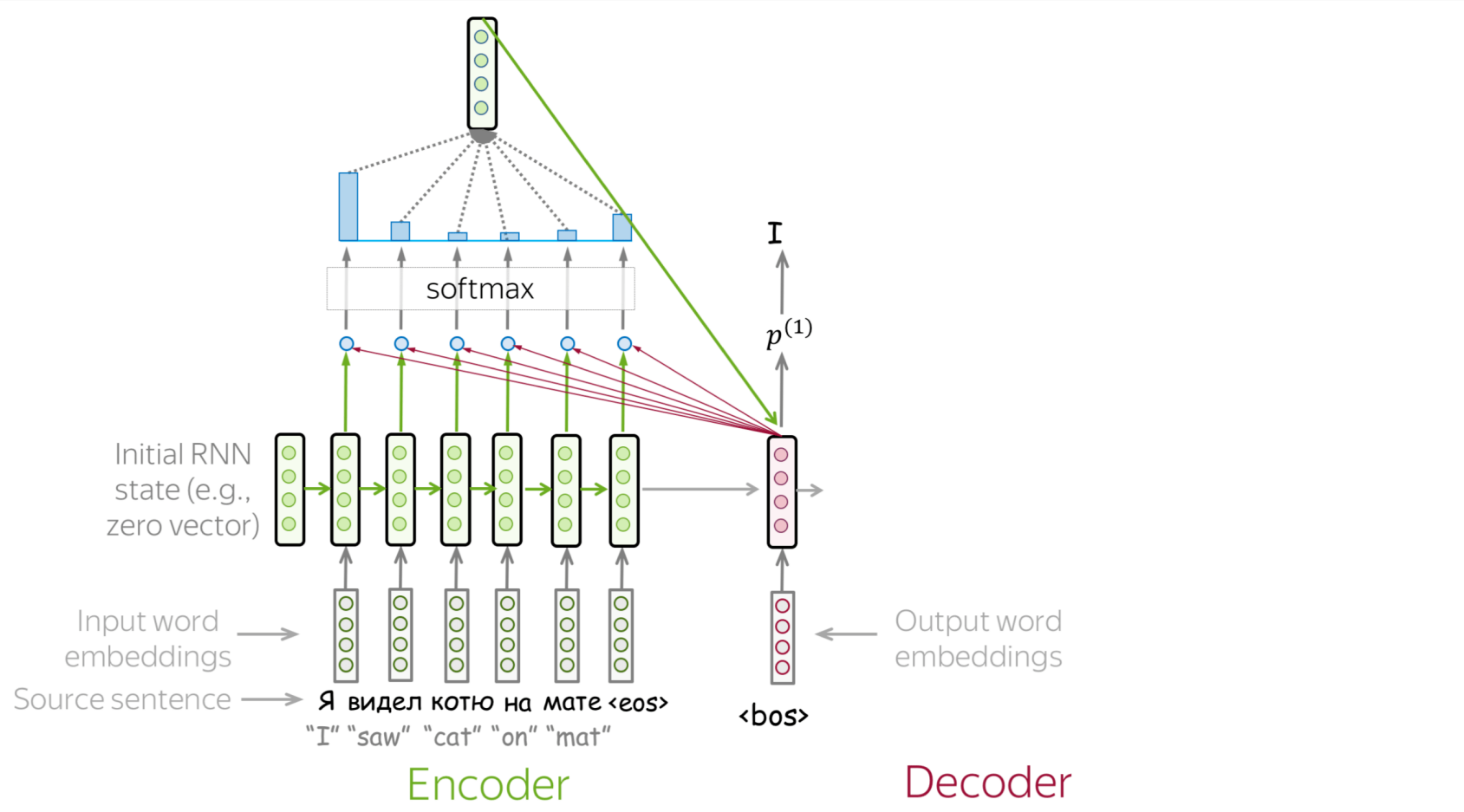
A better encoder-decoder model...

A better design allows the decoder to recompute the input's encoding as a function of the prefix of already-generated text, at each step:



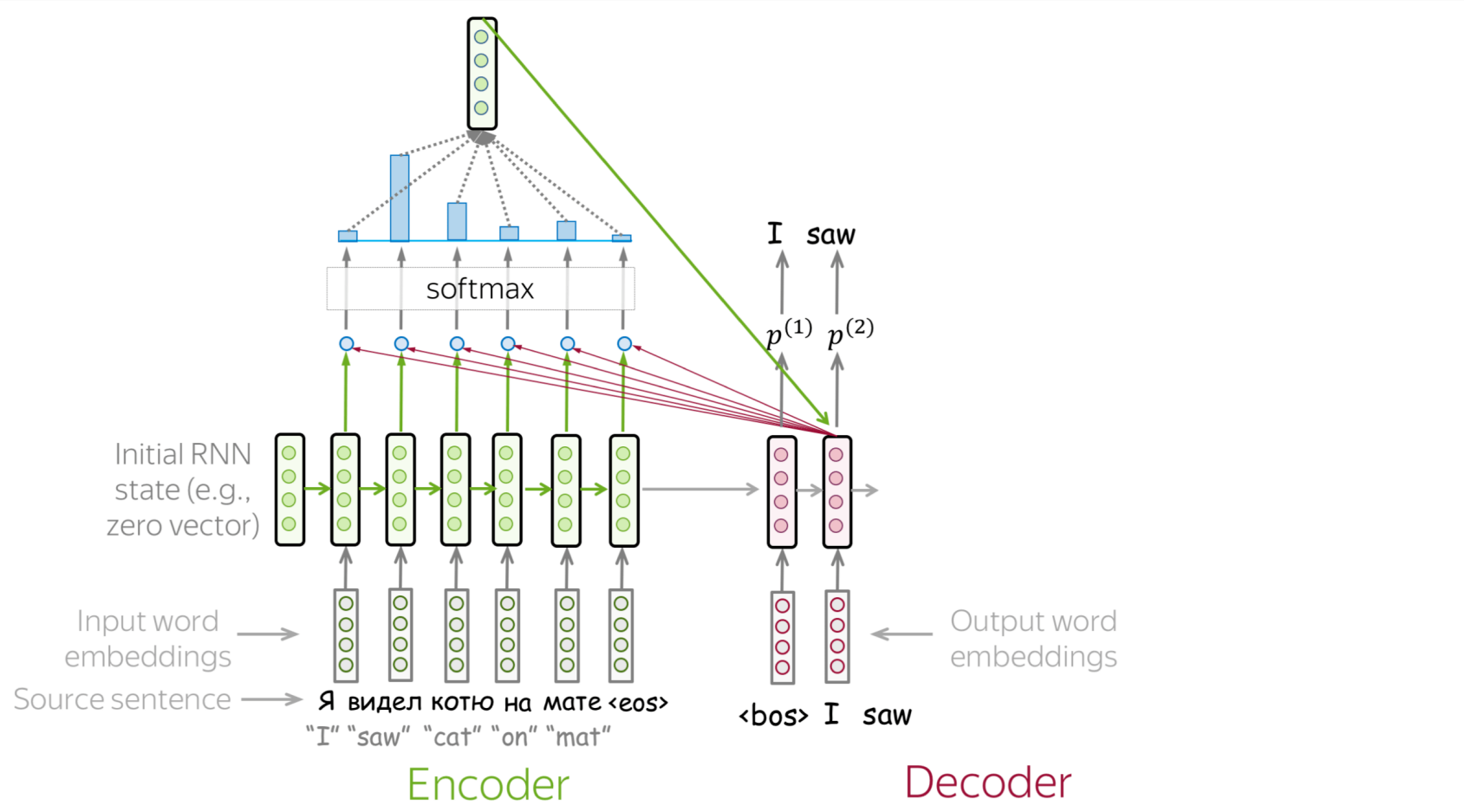
Encoder-Decoder with Attention

An **attention mechanism** allows the decoder to recombine the encoder's internal states in a weighted average whose coefficients depend on the decoder's internal state at each step:



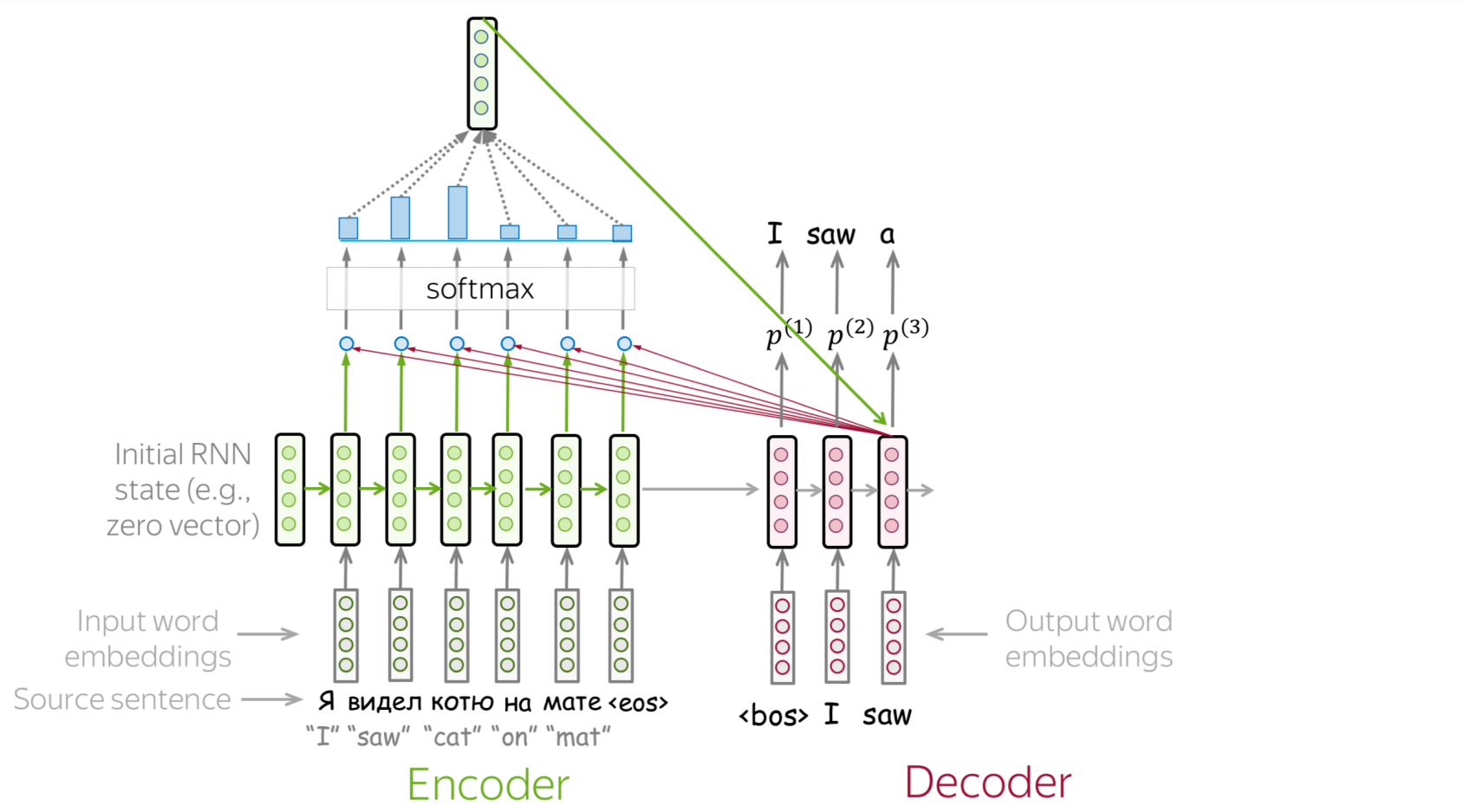
Encoder-Decoder with Attention

An **attention mechanism** allows the decoder to recombine the encoder's internal states in a weighted average whose coefficients depend on the decoder's internal state at each step:



Encoder-Decoder with Attention

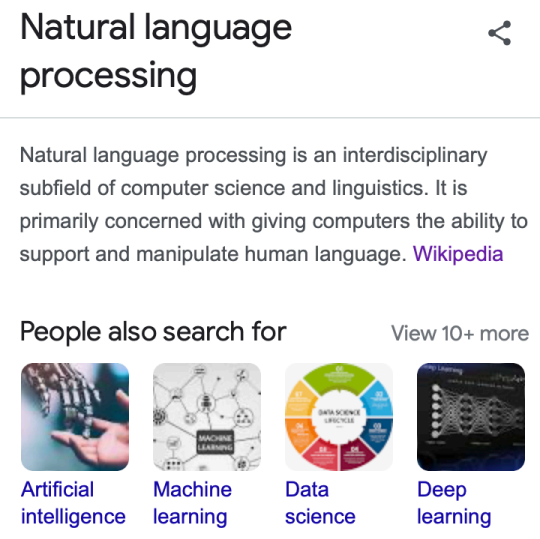
An **attention mechanism** allows the decoder to recombine the encoder's internal states in a weighted average whose coefficients depend on the decoder's internal state at each step:

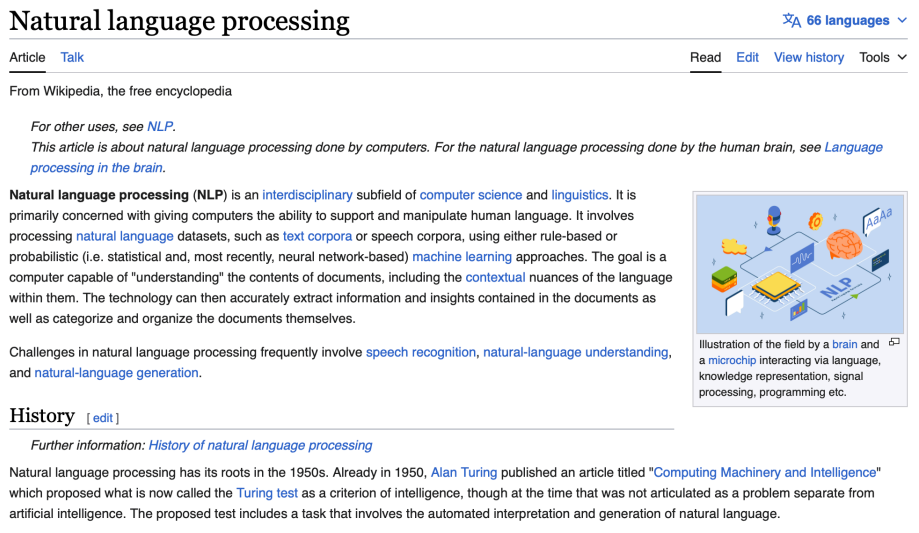


Text Summarisation

NLP task: generate a short version of a (collection of) document(s) that contains the most important information.

- Extractive: identify and extract important/relevant sentences from the input document(s).
- Abstractive: interpret the content of the document and generate an 'original' summary.





A neural abstractive summarisation model

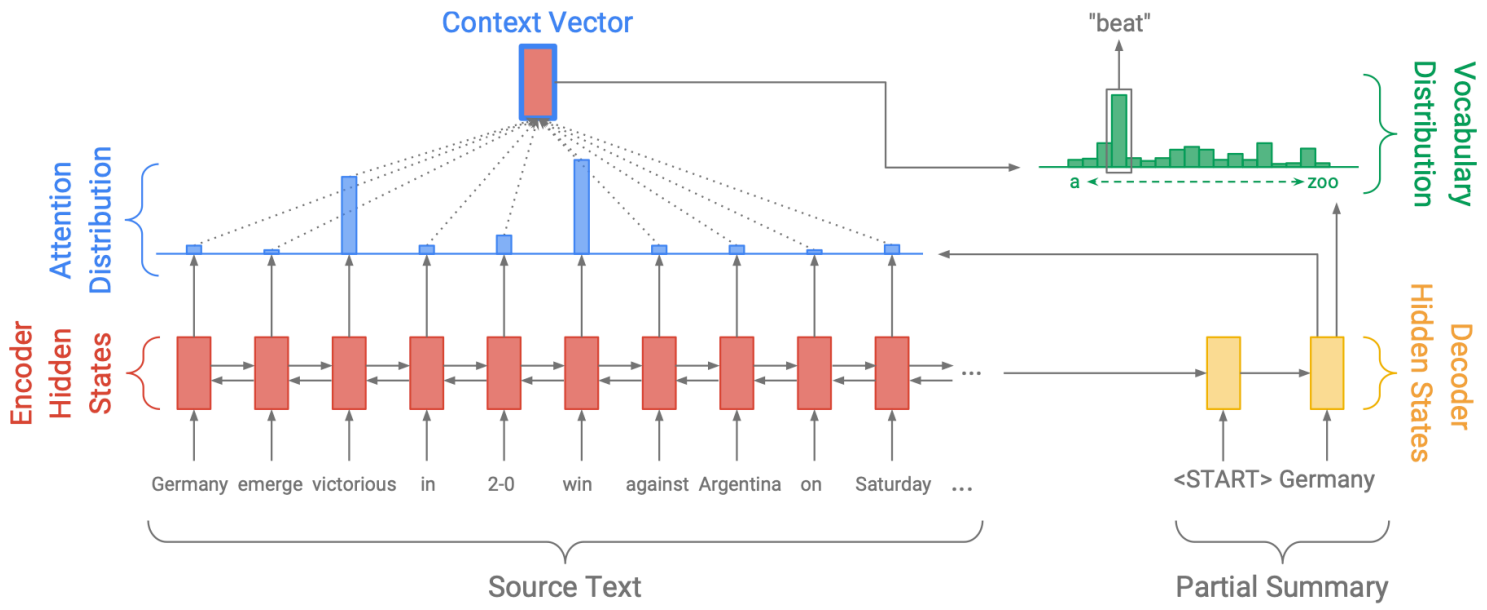


Figure 2: Baseline sequence-to-sequence model with attention. The model may attend to relevant words in the source text to generate novel words, e.g., to produce the novel word *beat* in the abstractive summary *Germany* ***beat*** *Argentina 2-0* the model may attend to the words *victorious* and *win* in the source text.

Figure from [12]

Parameter estimation

Data and Task

Data a collection of pairs (x, y) where both x and y can be treated as a sequence of outcomes from small discrete sets.

Statistical task observe x and predict a conditional distribution over all possible sequences.

NLP task map a sequence x to a sequence y: for example via $\arg\max_{y \in \mathcal{Y}} P(Y = y | X = x)$.

Formalisation of statistical task

Statistical model let the function $\mathbf{g}$ map from x to a chain rule factorisation of the conditional distribution $Y|X = x$:

$$Y_j|X = x, Y_{<j} = y_{<j} \sim \mathrm{Cat}(\mathbf{g}(x, y_{<j}; \theta)) \tag{4}$$

θ collectively refers to all trainable parameters in the model.

Formalisation of statistical task

Statistical model let the function $\mathbf{g}$ map from x to a chain rule factorisation of the conditional distribution $Y|X = x$:

$$Y_j|X = x, Y_{<j} = y_{<j} \sim \mathrm{Cat}(\mathbf{g}(x, y_{<j}; \theta)) \tag{4}$$

θ collectively refers to all trainable parameters in the model.

Statistical objective maximum likelihood of model given a dataset of observations $\mathcal{D}$: [see animation by Lena]

$$\mathcal{L}(\theta|\mathcal{D}) = \sum_{(x,y)\in\mathcal{D}} \log \underbrace{P(Y = y|X = x)}_{f(y|x;\theta)}$$

Formalisation of statistical task

Statistical model let the function $\mathbf{g}$ map from x to a chain rule factorisation of the conditional distribution $Y|X = x$:

$$Y_j|X = x, Y_{<j} = y_{<j} \sim \mathrm{Cat}(\mathbf{g}(x, y_{<j}; \theta)) \tag{4}$$

θ collectively refers to all trainable parameters in the model.

Statistical objective maximum likelihood of model given a dataset of observations $\mathcal{D}$: [see animation by Lena]

$$\begin{aligned} \mathcal{L}(\theta|\mathcal{D}) &= \sum_{(x,y)\in\mathcal{D}} \log \underbrace{P(Y = y|X = x)}_{f(y|x;\theta)} \\ &= \sum_{(x,y)\in\mathcal{D}} \underbrace{\sum_{j=1}^{|y|} \log g_{y_j}(x, y_{<j}; \theta)}_{\log f(y|x;\theta)} \end{aligned} \tag{5}$$

Formalisation of statistical task

Statistical model let the function $\mathbf{g}$ map from x to a chain rule factorisation of the conditional distribution $Y|X = x$:

$$Y_j|X = x, Y_{<j} = y_{<j} \sim \mathrm{Cat}(\mathbf{g}(x, y_{<j}; \theta)) \tag{4}$$

θ collectively refers to all trainable parameters in the model.

Statistical objective maximum likelihood of model given a dataset of observations $\mathcal{D}$: [see animation by Lena]

$$\begin{aligned} \mathcal{L}(\theta|\mathcal{D}) &= \sum_{(x,y)\in\mathcal{D}} \log \underbrace{P(Y = y|X = x)}_{f(y|x;\theta)} \\ &= \sum_{(x,y)\in\mathcal{D}} \underbrace{\sum_{j=1}^{|y|} \log g_{y_j}(x, y_{<j}; \theta)}_{\log f(y|x;\theta)} \end{aligned} \tag{5}$$

Algorithm For concave $\mathcal{L}$ (or convex negative log-likelihood), find θ such that $\nabla_\theta \mathcal{L}(\theta|\mathcal{D}) = \mathbf{0}$.

Parameter estimation

Algorithm Solve the equation $\nabla_\theta \mathcal{L}(\theta|\mathcal{D}) = \mathbf{0}$ for θ. There is no closed form solution. But an optimum can be found via a fixed-point iteration: $\theta \leftarrow \theta + \gamma \nabla_\theta \mathcal{L}(\theta|\mathcal{D})$ for $\gamma > 0$.

In practice we take stochastic gradient steps using subsets of $\mathcal{D}$ ('mini batches') each time. Also, in practice, we use a sophisticated optimiser (e.g., Adam) with dynamic learning rates.

A soup of names

You probably heard of the *cross entropy loss*, which is identical to the *negative* of the quantity in the previous slide.

Some people will also call it the categorical cross entropy loss, or the softmax loss.

‘Softmax loss’ is a bit odd, softmax is a vector-valued function, it’s hard to imagine it as a loss.

Categorical cross entropy is clear, cross entropy can be clear enough in context.

Predictions

Making decisions

Our final job, as modellers, is to find a reasonable way to form predictions.

That is, given an input x, our model outputs a representation of an entire probability distribution $P_{Y|X=x}$ (i.e., over all of $\mathcal{Y}$).

We are now confronted with the task to map from $P_{Y|X=x}$ to a single output y. This is often formulated as a search, or discrete optimisation, problem.

Most probable output

A common algorithm for making decisions is to search for the candidate output c which is assigned highest probability:

$$y^\star = \arg\max_{c \in \mathcal{Y}} f(c|x;\theta) \tag{6}$$

This can also be done in log space: $\arg\max_{c \in \mathcal{Y}} \log f(c|x;\theta)$.

Most probable output

A common algorithm for making decisions is to search for the candidate output c which is assigned highest probability:

$$y^\star = \arg\max_{c \in \mathcal{Y}} f(c|x;\theta) \tag{6}$$

This can also be done in log space: $\arg\max_{c \in \mathcal{Y}} \log f(c|x;\theta)$.

This is intractable for most models (incl. autoregressive models)! Common approximations include *greedy decoding* (iteratively argmax each step) or *beam search decoding* (maintain a fixed number of high-scoring candidates; see animation by Lena).

Most 'useful' in expectation [optional content]

Let $u(y, c; x)$ quantify the utility of c when y is known to be a valid output for x.

In decision theory, a rational decision maker acts by maximising expected utility under the model:

$$y^\star = \arg\max_{c \in \mathcal{Y}} \mathbb{E}[u(Y, c; x)] \tag{7}$$

Expected utility can be approximated via Monte Carlo (MC):

$$\mathbb{E}[u(Y, c; x)] \overset{\text{MC}}{\approx} \frac{1}{K} \sum_{k=1}^{K} u(y^{(k)}, c; x) \tag{8}$$

with $y^{(k)} \sim P_{Y|X=x}$. See [2, 3].

Enumerating candidates

- Greedy decoding
- Beam search
- Ancestral sampling
- Top-p and top-k sampling [6]
- Sampling without replacement [8]

Evaluation

Statistical: does the model fit the data well?

- perplexity
- statistics of model samples e.g., [5]

Task-driven: does the model support good decisions (in a benchmark)?

- exact-match/precision/recall/F1 for short generations (QA, entity linking, information extraction)
- string similarity (e.g., BLEU, ROUGE, METEOR, BEER)
- semantic similarity (e.g., COMET, BLEURT)

Design Choices

Architectures

Many choices of encoders and decoders:

- CNNs [4]
- Transformers [14]

The Tranformer, in particular, is today's architecture of choice.

Limitations

Architectures

Some architectures suffer from certain bottlenecks that may inadvertently constrain the quality of the features that can be learned:

- CNN models have strict context windows;
- RNN models work with a recurrent but narrow representation bottleneck.

Others trade bottlenecks for some other limitation:

- Transformers do not have a recurrent bottleneck, but i) have a larger memory footprint, ii) have a fixed computation depth, and iii) are notoriously data-hungry.

Availability of data

Sequence-to-sequence models are used to approach tasks that require sophisticated language understanding and language generation skills.

While NN encoders and decoders can *in principle* learn data representations that power generators that work *as if* they had such skills, achieving this kind of performance requires **a lot of data**.

Some tasks enjoy a lot of data (e.g., translation amongst some European languages), but many tasks do not (e.g., involving languages with fewer speakers and/or lesser online presence, tasks that require sophisticated annotation processes, etc.)

Quality of data

In a push to obtain ever more data, we often lose track of the quality and representativeness of the data we have.

Large datasets crawled from the web may distort world views, propagate harmful biases, echo the under-representation of social minorities, etc.

NN models learn data-driven representations of data, these may often power predictions that seem *as if* the model has some linguistic ability, but a good researcher is careful not to get carried away by such appearances and keeps a healthy dose of skepticism wrt what models can/cannot do and how to conceive of these functionalities.

What next?

- Don't miss E5 (join a seminar if you want to work on it with your peers and TAs).
- Homework: work through the puzzle at the end of this slide deck (and feel free to post questions on Ed).
- Next week we look into how to benefit from datasets for multiple tasks, even made-up tasks, in order to learn encoders and decoders that are more generally useful.

Puzzle

Puzzle

Consider our typical sequence-to-sequence model, that is, a neural parameterisation of a chain rule factorisation of the joint distribution of our output random sequence Y given an outcome of an input random sequence $X = x$.

Suppose the output random sequence has length J. The probability of any outcome $\langle y_1, \ldots, y_J \rangle$ is given by

$$P(Y = \langle y_1, \ldots, y_J \rangle | X = x) = \prod_{j=1}^{J} P(Y_j = y_j | X = x, Y_{<j} = y_{<j}) \tag{9}$$

Suppose we obtain an output sequence but the kth step is missing. How can we generate outcomes for Y_k given the assignments of all other variables?

It is sufficient to solve this for an example: $X = x$, and $\langle Y_1 = \text{the}, Y_3 = \text{dog}, Y_4 = \text{EOS} \rangle$, with missing Y_2.

Puzzle - Solution

Let's denote by O the set of output random variables we observe (that is, $\{Y_1, Y_3, Y_4\}$ in the example) and o their observed values (that is, $\{\text{the}, \text{dog}, \text{EOS}\}$), and by U the set of variables that are unobserved (that is $\{Y_2\}$ in the example). We want to express the probability that U takes on some value u (e.g., $Y_2 = w$ for any word w in the vocabulary, in one case w might be the word 'cute' for example) given the assignments of the observed variables O and $X = x$.

We start by application of the definition of conditional probability:

$$P(U = u | O = o, X = x) = \frac{P(O = o, U = u | X = x)}{P(O = o | X = x)} \tag{10}$$

Puzzle - Solution

Note that the numerator is exactly the joint distribution we have access to. In our example: $P(O = o, U = u|X = x) = P(\underbrace{Y_1 = \text{the}, Y_3 = \text{dog}, Y_4 = \text{EOS}}_{O=o}, \underbrace{Y_2 = w}_{U=u}|X = x)$.

Note the the denominator is the marginal of the numerator, where we marginalise out all possible assignments of U. In our example, we would marginalise out all possibilities for the second token. If $\mathcal{V}$ is the entire vocabulary, we would compute $\sum_{t \in \mathcal{V}} P(Y_1 = \text{the}, Y_2 = t, Y_3 = \text{dog}, Y_4 = \text{EOS}|X = x)$.

Suppose in general an output sequence has length J and the vocabulary has size V. What's the computational complexity of evaluating the conditional probability of an assignment of some Y_k given everything else?

Puzzle - Solution

Probabilities that condition on past context are simple because those are directly predicted by our NN (provided we have access to observations for all variables in the past). Probabilities that condition on future are much more difficult because we need to assess the joint probability for every possible assignment of the unobserved variable (in the denominator of conditional probability).

Each joint probability takes J calls to our NN $\mathbf{g}(\cdot; \theta)$ (one per token in the sequence). We have to perform this computation V times, once per possible value of Y_k. So the total computation takes time proportional to $\mathcal{O}(JV)$.

References

[1] Nicola De Cao, Gautier Izacard, Sebastian Riedel, and Fabio Petroni. Autoregressive entity retrieval. In *International Conference on Learning Representations*, 2021. URL *https://openreview.net/forum?id=5k8F6UU39V*.

[2] Bryan Eikema and Wilker Aziz. Is MAP decoding all you need? the inadequacy of the mode in neural machine translation. In *Proceedings of the 28th International Conference on Computational Linguistics*, pages 4506–4520, Barcelona, Spain (Online), December 2020. International Committee on Computational Linguistics. doi: 10.18653/v1/2020.coling-main.398. URL *https://aclanthology.org/2020.coling-main.398*.

[3] Bryan Eikema and Wilker Aziz. Sampling-based minimum Bayes risk decoding for neural machine translation. In *EMNLP*, 2022.

[4] Jonas Gehring, Michael Auli, David Grangier, and Yann Dauphin. A convolutional encoder model for neural machine translation. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 123–135, Vancouver, Canada, July 2017. Association for Computational Linguistics. doi: 10.18653/v1/P17-1012. URL https://aclanthology.org/P17-1012.

[5] Mario Giulianelli, Joris Baan, Wilker Aziz, Raquel Fernández, and Barbara Plank. What comes next? evaluating uncertainty in neural text generators against human production variability. In *EMNLP*, 2023.

[6] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. *arXiv preprint arXiv:1904.09751*, 2019.

[7] Martin Josifoski, Nicola De Cao, Maxime Peyrard, Fabio Petroni, and Robert West. GenIE: Generative information extraction. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4626–4643, Seattle, United States, July 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.naacl-main.342. URL https://aclanthology.org/2022.naacl-main.342.

[8] Wouter Kool, Herke Van Hoof, and Max Welling. Stochastic beams and where to find them: The gumbel-top-k trick for sampling sequences without replacement. In *International Conference on Machine Learning*, pages 3499–3508. PMLR, 2019.

[9] Guillaume Lample, Miguel Ballesteros, Sandeep Subramanian, Kazuya Kawakami, and Chris Dyer. Neural architectures for named entity recognition. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 260–270, San Diego, California, June 2016. Association for Computational Linguistics. doi: 10.18653/v1/N16-1030. URL https://aclanthology.org/N16-1030.

[10] Wang Ling, Chris Dyer, Alan W Black, Isabel Trancoso, Ramón Fermandez, Silvio Amir, Luís Marujo, and Tiago Luís. Finding function in form: Compositional character models for open vocabulary word representation. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pages 1520–1530, Lisbon, Portugal, September 2015. Association for Computational Linguistics. doi: 10.18653/v1/D15-1176. URL https://aclanthology.org/D15-1176.

[11] Alexander M. Rush, Sumit Chopra, and Jason Weston. A neural attention model for abstractive sentence summarization. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pages 379–389, Lisbon, Portugal, September 2015. Association for Computational Linguistics. doi: 10.18653/v1/D15-1044. URL https://aclanthology.org/D15-1044.

[12] Abigail See, Peter J. Liu, and Christopher D. Manning. Get to the point: Summarization with pointer-generator networks. In Regina Barzilay and Min-Yen Kan, editors, *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1073–1083, Vancouver, Canada, July 2017. Association for Computational Linguistics. doi: 10.18653/v1/P17-1099. URL https://aclanthology.org/P17-1099.

[13] Ilya Sutskever, Oriol Vinyals, and Quoc V. V Le. Sequence to sequence learning with neural networks. In Z. Ghahramani, M. Welling, C. Cortes, N.D. Lawrence, and K.Q. Weinberger, editors, *NIPS, 2014*, pages 3104–3112. Montreal, Canada, 2014.

[14] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, pages 6000–6010, 2017. URL *http://papers.nips.cc/paper/7181-attention-is-all-you-need*.